

Name:

Subject: Javascript Programme

Date:

Q.1>Demonstrate Singly and Doubly Linked List.

```
class Node
{
  constructor(element)
  {
    this.element=element;
    this.next=null;
  }
}
class LinkedList
{
  constructor()
  {
    this.head=null;
    this.size=0;
  }
  add(element)
  {
    var temp=new Node(element);
    var temp1;
    if(this.head==null)
      this.head=temp;
    else
    {
      temp1= this.head;
      while(temp1.next)
      {
        temp1=temp1.next
      }
      temp1.next=temp;
    }
    this.size++;
  }
  insertAt(element, index)
  {
    if(index>0 && index>this.size)
      return false;
    else
    {
      var temp=new Node(element);
      var temp1, temp2;
      temp1=this.head;
      if (index==0)
      {
        this.next=this.head;
        this.head=temp;
      }
      else
      {
        temp1=this.head;
        var it=0;
        while(it<index)
        {
          it++;
          temp2=temp1;
          temp1=temp1.next;
        }
        temp.next = temp1;
        temp2.next = temp;
      }
      this.size++;
    }
  }
}
```

```

PrintList()
{
    var temp1=this.head;
    while(temp1)
    {
        console.log(temp1.element);
        temp1=temp1.next;
    }
}
findPrevious(item)
{
    var currNode = this.head;
    while (!(currNode.next == null) &&(currNode.next.element != item))
    {
        currNode = currNode.next;
    }
    return currNode;
}
remove(item)
{
    var prevNode = this.findPrevious(item);
    if (!(prevNode.next == null))
    {
        prevNode.next = prevNode.next.next;
    }
}
}
ll =new LinkedList();
ll.add(10);
ll.add(20);
ll.add(30);
ll.insertAt(50,1);
console.log(ll.insertAt(100,6));
ll.PrintList();
console.log("after removing the given element, the result is :")
ll.remove(30);
ll.PrintList();

```

Output:

```

false
10
50
20
30
after removing the given element, the result is :
10
50
20

```

Q.2. STACK implementation using Array with PUSH, POP operations

```

class Stack {
    constructor() {
        this.items = [];
    }

    // add element to the stack
    add(element) {
        return this.items.push(element);
    }

    // remove element from the stack
    remove() {
        if(this.items.length > 0) {
            return this.items.pop();
        }
    }

    // view the last element
    peek() {
        return this.items[this.items.length - 1];
    }

    // check if the stack is empty
    isEmpty(){
        return this.items.length == 0;
    }
}

```

```

        // the size of the stack
        size(){
            return this.items.length;
        }

        // empty the stack
        clear(){
            this.items = [];
        }
    }

    let stack = new Stack();
    stack.add(1);
    stack.add(2);
    stack.add(4);
    stack.add(8);
    console.log(stack.items);

    stack.remove();
    console.log(stack.items);

    console.log(stack.peek());

    console.log(stack.isEmpty());

    console.log(stack.size());

    stack.clear();
    console.log(stack.items);

```

Output:

```

[ 1, 2, 4, 8 ]
[ 1, 2, 4 ]
4
false
3
[]

```

Q.3:Reverse a String using Stack.

```

class Stack {
    constructor() {
        this.top = null
    }
    push(ele) {
        var node = new newNode(ele)
        node.next = this.top
        this.top = node
    }
    pop() {
        var temp = this.top
        var char = temp.data
        this.top = this.top.next
        temp = null
        return char
    }
    reverseString(str) {
        var i = 0
        var reversestr = ""
        while (i != str.length) {
            this.push(str.charAt(i))
            i++
        }
        var temp = this.top
        while (temp != null) {
            var char
            char = this.pop()
            reversestr += char
            temp = this.top
        }
        return reversestr
    }
    display() {
        var temp = this.top
        while (temp != null) {
            console.log(temp.data)
            temp = temp.next
        }
    }
}

```

```

    }
  }
}
class newNode {
  constructor(data, next) {
    this.data = data
    this.next = null
  }
}

const stack = new Stack()
const string = "pratik"
const reverse = stack.reverseString(string)
console.log(`The String provided - ${string}\nString in reverse format -${reverse}`)

```

Output:

```

The String provided - pratik
String in reverse format -kitarp

```

Q.4>Check for Balance Parantheses by using Stacks.

```

// Javascript program for checking
// balanced brackets

// Function to check if brackets are balanced
function areBracketsBalanced(expr)
{
    // Using ArrayDeque is faster
    // than using Stack class
    let stack = [];

    // Traversing the Expression
    for(let i = 0; i < expr.length; i++)
    {
        let x = expr[i];

        if (x == '(' || x == '[' || x == '{')
        {
            // Push the element in the stack
            stack.push(x);
            continue;
        }

        // If current character is not opening
        // bracket, then it must be closing.
        // So stack cannot be empty at this point.
        if (stack.length == 0)
            return false;

        let check;
        switch (x){
            case ')':
                check = stack.pop();
                if (check == '{' || check == '[')
                    return false;
                break;

            case '}':
                check = stack.pop();
                if (check == '(' || check == '[')
                    return false;
                break;

            case ']':
                check = stack.pop();
                if (check == '(' || check == '{')
                    return false;
                break;
        }
    }

    // Check Empty Stack
    return (stack.length == 0);
}

```

```
// Driver code
let expr = "([{}])";

// Function call
if (areBracketsBalanced(expr))
    document.write("Balanced");
    //write("Balanced ");
else
    document.write("Not Balanced ");
```

Output:

Balanced

Q.5>Implement stack Using Linkelist

```
class Stack
{
    constructor ()
    {
        this.top = null
    }

    push(ele)
    {
        var node=new newNode(ele)
        console.log(node)
        node.next=this.top
        this.top=node
    }

    pop()
    {
        var temp=this.top
        this.top=this.top.next
        temp=null
    }

    display()
    {
        var temp=this.top
        while(temp!=null)
        {
            console.log(temp.data)
            temp=temp.next
        }
    }
}

class newNode
{
    constructor (data, next)
    {
        this.data = data
        this.next = null
    }
}

const top = new Stack()
top.push(11)
top.push(22)
top.push(33)
top.push(44)
top.push(55)
top.push(66)
console.log("After PUSH...")
top.display()
top.pop()
console.log("After POP...")
top.display()
top.pop()
console.log("After POP...")
top.display()
```

Output:

```
newNode { data: 11, next: null }
newNode { data: 22, next: null }
newNode { data: 33, next: null }
newNode { data: 44, next: null }
newNode { data: 55, next: null }
newNode { data: 66, next: null }
After PUSH...
66
55
44
33
22
11
After POP...
55
44
33
22
11
After POP...
44
33
22
11
```

Q.6>Demonstration of Linear Queue,Circular Queue,Priority Queue.

a>Linear Queue:

```
function queue()
{
  this.data=[];
  this.rear=0;
  this.front=0;
  this.enqueue=enqueue;
  this.MaxSize=10;
  this.print=print;
  this.length=length;
  this.isEmpty=isEmpty;
  this.dequeue=dequeue;
}
function enqueue(element)
{
  if(this.rear<this.MaxSize)
  {
    this.data[this.rear++]=element; //data[0]=10
  }
}

function print()
{
  for(let i=0;i<this.data.length;i++)
  {
    console.log(this.data[i])
  }
}
function length()
{
  return this.rear;
}
function isEmpty()
{
  return this.rear=0;
}
function dequeue()
{
  if(this.isEmpty()==false)
```

```

    {
        this.front=this.front+1;
        return this.data.shift();
    }
}

var q=new queue();
q.enqueue(10);
q.enqueue(20);
q.enqueue(30);

console.log("elements in queue :");
q.print();
console.log("length of queue :",q.length());
console.log("delete one element of queue is :")
console.log(q.dequeue())
console.log("elements in queue :");
q.print();

```

Output:

```

elements in queue :
10
20
30
length of queue : 3
delete one element of queue is :
10
elements in queue :
20
30

```

b>Circular Queue:

```

class CircularQueue
{
    constructor(size)
    {
        this.data=[];
        this.size=size;
        this.length=0;
        this.front=0;
        this.rear=-1;
    }
    isEmpty()
    {
        return (this.length==0)
    }
    enqueue(element)
    {
        if(this.length>=this.size)
            console.log("full");
        //rear=(rear+1)%size;
        this.data[(this.rear+1)%this.size] =element;// data[0]=10
        this.length++;
    }

    print()
    {
        for(let i=0;i<this.data.length;i++)
        {
            console.log[this.data[i]];
        }
    }

    getfront()
    {
        if(this.isEmpty())
        {
            console.log("no element in circular queue");
        }
        return this.data[(this.front) % (this.size)];
    }
}

```

dequeue()

```

    {
        if(this.isEmpty())
            console.log("no element");
        const value= this.getfront();
        this.data[this.front%this.size]=null;
        this.front++;
        this.length--;
        console.log(value);
    }
}
cq=new CircularQueue(5);
cq.enqueue(10);
cq.enqueue(15);
cq.enqueue(16);
cq.enqueue(17);
cq.enqueue(18);
console.log("deleted element ");
cq.dequeue();

```

Output:

```

deleted element
18

```

c>Priority Queue:

```

class QElement {
    constructor(element, priority)
    {
        this.element = element;
        this.priority = priority;
    }
}
class PriorityQueue {

    constructor()
    {
        this.items = [];
    }

    enqueue(element, priority)
    {
        var qElement = new QElement(element, priority);
        var contain = false;
        for (var i = 0; i < this.items.length; i++) {
            if (this.items[i].priority > qElement.priority) {
                this.items.splice(i, 0, qElement);
                contain = true;
                break;
            }
        }

        if (!contain) {
            this.items.push(qElement);
        }
    }

    dequeue()
    {
        if (this.isEmpty())
            return "Underflow";
        return this.items.shift();
    }

    front()
    {
        if (this.isEmpty())
            return "No elements in Queue";
        return this.items[0];
    }

    rear()
    {
        if (this.isEmpty())
            return "No elements in Queue";
        return this.items[this.items.length - 1];
    }

    isEmpty()
    {
        return this.items.length == 0;
    }
}

printPQueue()

```



```

    {
        var str = "";
        for (var i = 0; i < this.items.length; i++)
            str += this.items[i].element + " ";
        return str;
    }
}
var priorityQueue = new PriorityQueue();

console.log("Is Queue Empty :",priorityQueue.isEmpty());

console.log(priorityQueue.front());

priorityQueue.enqueue("Pooja", 2);
priorityQueue.enqueue("Aditi", 1);
priorityQueue.enqueue("Atharva", 1);
priorityQueue.enqueue("Madhura", 2);
priorityQueue.enqueue("Devashree", 3);

console.log("Elements of Priority Queue :");
console.log(priorityQueue.printPQueue());

console.log("Front element:",priorityQueue.front().element);

console.log("Rear element :",priorityQueue.rear().element);

console.log("Deleting element :",priorityQueue.dequeue().element);

priorityQueue.enqueue("Sachin", 2);
console.log("New Elements of Priority Queue:");
console.log(priorityQueue.printPQueue());

```

Output:

```

Is Queue Empty : true
No elements in Queue
Elements of Priority Queue :
Aditi Atharva Pooja Madhura Devashree
Front element: Aditi
Rear element : Devashree
Deleting element : Aditi
New Elements of Priority Queue:
Atharva Pooja Madhura Sachin Devashree

```

Q.7>Reverse Stack using Queue.

```

//Reversing Stack using Queue
class Queue
{
    constructor ()
    {
        this.front = null
        this.rear = null
    }

    insert(ele)
    {
        var node = new newNode(ele)
        if(this.front==null)
            this.front=node
        else
            this.rear.next=node
        this.rear=node
    }
    //Get from queue and push on stack
    push_stack()
    {
        var temp=this.front
        while(temp!=null)
        {
            top.push(temp.data)
            temp=temp.next
            this.front=this.front.next
        }
    }
}
display()

```

```

        {
            var temp=this.front
            while(temp!=null)
            {
                console.log(temp.data)
                temp=temp.next
            }
        }
    }
}
class Stack
{
    constructor ()
    {
        this.top = null
    }

    push(ele)
    {
        var node=new newNode(ele)
        node.next=this.top
        this.top=node
    }
    //pop from stack and add to queue
    pop_add_queue()
    {
        var temp=this.top
        while(temp!=null)
        {
            front.insert(temp.data)
            temp=temp.next
        }
        this.top=null
    }
    display()
    {
        var temp=this.top
        while(temp!=null)
        {
            console.log(temp.data)
            temp=temp.next
        }
    }
}

class newNode
{
    constructor (data, next)
    {
        this.data = data
        this.next = null
    }
}

const top = new Stack()
const front = new Queue()
top.push(10)
top.push(20)
top.push(30)
top.push(40)
top.push(50)
top.push(60)
console.log("After PUSH...")
top.display()
top.pop_add_queue()
console.log("After adding to Queue...")
front.display()
front.push_stack()
console.log("After REVERSE...")
top.display()

```

Output:

```

After PUSH...
60
50
40
30

```

```
20
10
After adding to Queue...
```

```
60
50
40
30
20
10
```

After REVERSE...

```
10
20
30
40
50
60
```

Q.8>Practical based on binary Search tree implementation with its operations.

//Binary Search Tree with insertNode, removeNode and traversals

```
class BinarySearchTree
{
    constructor()
    {
        // root of a binary search tree
        this.root = null
    }
    getrootNode()
    {
        return this.root
    }

    insert(data)
    {
        var newNode = new Node(data)
        if(this.root == null)
            this.root = newNode
        else
            this.insertNode(this.root, newNode)
    }

    insertNode(node, newNode)
    {
        if(newNode.data < node.data)
        {
            // if left is null insert node here
            if(node.left == null)
                node.left = newNode
            else
                this.insertNode(node.left, newNode)
        }

        else
        {
            // if right is null insert node here
            if(node.right == null)
                node.right = newNode
            else
                this.insertNode(node.right, newNode)
        }
    }
    remove(data)
    {
        this.root = this.removeNode(this.root, data);
    }

    removeNode(node, key)
    {
        if(node === null)
            return null;
```

```
        else if(key < node.data)
```

```

        {
            node.left = this.removeNode(node.left, key);
            return node;
        }
        else if(key > node.data)
        {
            node.right = this.removeNode(node.right, key);
            return node;
        }

        else
        {
            // deleting node with no children
            if(node.left === null && node.right === null)
            {
                node = null;
                return node;
            }

            // deleting node with one children
            if(node.left === null)
            {
                node = node.right;
                return node;
            }

            else if(node.right === null)
            {
                node = node.left;
                return node;
            }

            // Deleting node with two children
            // mininum node of the rigt subtree is stored in temp
            var temp = this.findMinNode(node.right);
            node.data = temp.data;
            node.right = this.removeNode(node.right, temp.data);
            return node;
        }
    }
    findMinNode(node)
    {
        // if left of a node is null
        // then it must be minimum node
        if(node.left === null)
            return node;
        else
            return this.findMinNode(node.left);
    }

    inorder(node)
    {
        //console.log(node.data)
        if(node != null)
        {
            this.inorder(node.left)
            console.log(node.data)
            this.inorder(node.right)
        }
    }
    preorder(node)
    {
        //console.log(node.data)
        if(node != null)
        {
            console.log(node.data)
            this.inorder(node.left)
            this.inorder(node.right)
        }
    }
    postorder(node)
    {
        //console.log(node.data)
        if(node != null)
        {
            this.inorder(node.left)
            this.inorder(node.right)
            console.log(node.data)
        }
    }
}

```

```

    }
}

class Node
{
    constructor (data, left, right)
    {
        this.data = data
        this.left = null
        this.right = null
    }
}

const bst=new BinarySearchTree()
var r=new BinarySearchTree()
bst.insert(55)
bst.insert(10)
bst.insert(89)
bst.insert(62)
bst.insert(104)
bst.insert(60)
bst.insert(5)
bst.insert(-10)
bst.insert(20)
r=bst.getrootNode()
console.log("Inorder traversal of tree..")
bst.inorder(r)
console.log("Preorder traversal of tree..")
bst.preorder(r)
console.log("Postorder traversal of tree..")
bst.postorder(r)
bst.remove(89)
console.log("Inorder traversal of tree..")
bst.inorder(r)
bst.remove(-10)
console.log("Inorder traversal of tree..")
bst.inorder(r)
console.log("Inorder traversal of tree..")
bst.inorder(r)
bst.remove(10)
console.log("Inorder traversal of tree..")
bst.inorder(r)

```

Output:

```

Inorder traversal of tree..
-10
5
10
20
55
60
62
89
104
Preorder traversal of tree..
55
-10
5
10
20
60
62
89
104
Postorder traversal of tree..
-10
5
10
20
60
62
89
104

```

```

55
Inorder traversal of tree..
-10
5
10
20
55
60
62
104
Inorder traversal of tree..
5
10
20
55
60
62
104
Inorder traversal of tree..
5
10
20
55
60
62
104
Inorder traversal of tree..
5
20
55
60
62
104

```

Q.9>Graph implementation and graph traversals.

```

class Graph {

    constructor(noOfVertices)
    {
        this.noOfVertices = noOfVertices
        this.AdjList = new Map()
    }

    // add vertex to the graph
    addVertex(v)
    {
        this.AdjList.set(v, [])
    }
    // add edge to the graph
    addEdge(v, w)
    {
        //add an edge from v to w also
        this.AdjList.get(v).push(w)

        // add an edge from w to v also
        this.AdjList.get(w).push(v)
    }
    // Prints the vertex and adjacency list
    printGraph()
    {
        // get all the vertices
        var get_keys = this.AdjList.keys()

        // iterate over the vertices
        for (var i of get_keys)
        {
            // great the corresponding adjacency list for the vertex
            var get_values = this.AdjList.get(i)
            var conc = ""

```

```

        // iterate over the adjacency list concatenate the values into a string

```

```

        for (var j of get_values)
            conc += j + " ";

        // print the vertex and its adjacency list
        console.log(i + " -> " + conc);
    }
}

bfs(startingNode)
{
    // create a visited object
    var visited = {};

    // Create an object for queue
    var q = new Queue();

    // add the starting node to the queue
    visited[startingNode] = true;
    q.enqueue(startingNode);

    // loop until queue is element
    while (!q.isEmpty())
    {
        // get the element from the queue
        var getQueueElement = q.dequeue();

        // passing the current vertex to callback funtion
        console.log(getQueueElement);

        // get the adjacent list for current vertex
        var get_List = this.AdjList.get(getQueueElement);

        // loop through the list and add the element to the
        // queue if it is not processed yet
        for (var i in get_List) {
            var neighbour = get_List[i];

            if (!visited[neighbour]) {
                visited[neighbour] = true;
                q.enqueue(neighbour);
            }
        }
    }
}

dfs(startingNode)
{
    var visited = {};
    this.DFS_traversal(startingNode, visited);
}

// Recursive function which process and explore
// all the adjacent vertex of the vertex with which it is called
DFS_traversal(vert, visited)
{
    visited[vert] = true;
    console.log(vert);

    var get_neighbours = this.AdjList.get(vert);

    for (var i in get_neighbours) {
        var get_elem = get_neighbours[i];
        if (!visited[get_elem])
            this.DFS_traversal(get_elem, visited);
    }
}

}

class Queue
{
    // Array is used to implement a Queue
    constructor()
    {
        this.items = [];
    }

    enqueue(element)
    {
        // adding element to the queue
    }
}

```

```

        this.items.push(element);
    }
    dequeue()
    {
        if(this.isEmpty())
            return "Underflow";
        return this.items.shift();
    }

    isEmpty()
    {
        // return true if the queue is empty.
        return this.items.length == 0;
    }
}

var g = new Graph(6);
var vertices = ['A','B','C','D','E','F'];
for (var i = 0; i < vertices.length; i++)
{
    g.addVertex(vertices[i]);
}

// adding edges
g.addEdge('A', 'B');
g.addEdge('A', 'D');
g.addEdge('A', 'E');
g.addEdge('B', 'C');
g.addEdge('D', 'E');
g.addEdge('E', 'F');
g.addEdge('E', 'C');
g.addEdge('C', 'F');
g.printGraph();
console.log("BFS");
g.bfs('A');
console.log("DFS");
g.dfs('A');

```

Output:

```

A -> B D E
B -> A C
C -> B E F
D -> A E
E -> A D F C
F -> E C
BFS
A
B
D
E
C
F
DFS
A
B
C
E
D

```

Q.10>Implementation of Hashing.

```

unction generateHash(string) {
    var hash = 0;
    if (string.length == 0)
        return hash;
    for (let i = 0; i < string.length; i++) {
        var charCode = string.charCodeAt(i);
        hash = ((hash << 7) - hash) + charCode;
        hash = hash & hash;
    }
}

```

```

// hash.substring(0, 5);
return hash;

```



```

}

var a=generateHash("abc");
//var quoteText = `"${a}"`
console.log(`"${a}"`);

```

Output:

```
"1577058"
```

Q.11>Practical based on Brute Force Technique.

//Practical based on Brute Force technique

```

class search
{
    linear_search (arr, key)
    {
        var len=arr.length;
        var flag=0;
        for(var i=0;i<len;i++)
        {
            if (arr[i]==key)
            {
                console.log("Element found!");
                flag=1;
                break;
            }
        }
        if(flag==0)
            console.log("Element not found!");
    }
}

// Execute Linear search
var s=new search();
var arr = [1, 3, 5, 7, 8, 9];
var key = 3;
s.linear_search(arr, key);

```

Output:

```
Element found!
```

Practical based on Greedy Algorithm-Prim's/Kruskal's algorithm.

```

/*
    create adjacency matrix for use in prims algorithm
    note: we could improve the running time of prims algorithm by
    implementing a priority queue data structure instead of a matrix
*/
function createAdjMatrix(V, G) {

    var adjMatrix = [];

    // create N x N matrix filled with 0 edge weights between all vertices
    for (var i = 0; i < V; i++) {
        adjMatrix.push([]);
        for (var j = 0; j < V; j++) { adjMatrix[i].push(0); }
    }
    console.log(G)

    // populate adjacency matrix with correct edge weights
    console.log("graph length="+G.length)
    for (var i = 0; i < G.length; i++) {

        adjMatrix[G[i][0]][G[i][1]] = G[i][2]; // [0] means start vertex and [1] end
        vertex and [2] means weights
        //console.log(adjMatrix[G[i][0]][G[i][1]])
        adjMatrix[G[i][1]][G[i][0]] = G[i][2];
    }
}

```

```

    return adjMatrix;
}

function prims(V, G) {

    // create adj matrix from graph
    var adjMatrix = createAdjMatrix(V, G);

    // arbitrarily choose initial vertex from graph
    var vertex = 0;

    // initialize empty edges array and empty MST
    var MST = [];
    var edges = [];
    var visited = [];
    var minEdge = [null,null,Infinity];

    // run prims algorithm until we create an MST
    // that contains every vertex from the graph
    while (MST.length !== V-1) {

        // mark this vertex as visited
        visited.push(vertex);

        // add each edge to list of potential edges
        for (var r = 0; r < V; r++) {
            if (adjMatrix[vertex][r] !== 0) {
                edges.push([vertex,r,adjMatrix[vertex][r]]);
            }
        }

        // find edge with the smallest weight to a vertex
        // that has not yet been visited
        for (var e = 0; e < edges.length; e++) {
            if (edges[e][2] < minEdge[2] && visited.indexOf(edges[e][1]) === -1) {
                minEdge = edges[e];
            }
        }

        // remove min weight edge from list of edges
        edges.splice(edges.indexOf(minEdge), 1);

        // push min edge to MST
        MST.push(minEdge);

        // start at new vertex and reset min edge
        vertex = minEdge[1];
        minEdge = [null,null,Infinity];

    }

    return MST;
}

// graph vertices are actually represented as numbers
// like so: 0, 1, 2, ... V-1
var a = 0, b = 1, c = 2, d = 3, e = 4, f = 5;

// graph edges with weights
// diagram of graph is shown above
var graph = [
    [a,b,2],
    [a,c,3],
    [b,d,3],
    [b,c,5],
    [b,e,4],
    [c,e,4],
    [d,e,2],
    [d,f,3],
    [e,f,5]
];

// pass the # of vertices and the graph to run prims algorithm
console.log(prims(6, graph));

```

Output:

```
[
  [ 0, 1, 2 ], [ 0, 2, 3 ],
  [ 1, 3, 3 ], [ 1, 2, 5 ],
  [ 1, 4, 4 ], [ 2, 4, 4 ],
  [ 3, 4, 2 ], [ 3, 5, 3 ],
  [ 4, 5, 5 ]
]
graph length=9
[ [ 0, 1, 2 ], [ 0, 2, 3 ], [ 1, 3, 3 ], [ 3, 4, 2 ], [ 3, 5, 3 ] ]
```

Q.13>Practical based on Divide and Conquer Technique-Binary Search,Tower of Hanoi.

```
//Binary Search
class search
{
    binary_search (arr, key)
    {
        var low=0, high=arr.length-1;
        var flag=0;

        while (low<=high)
        {
            // Find the mid index
            var mid=Math.floor((low+high)/2);
            // If element is present at mid, return True
            if (arr[mid]==key)
            {
                console.log("Element found!");
                flag=1;
                break;
            }
            // Else look in left or right half accordingly
            else if (arr[mid] < key)
                low = mid + 1;
            else
                high = mid - 1;
        }
        if(flag==0)
            console.log("Element not found!");
    }
}

// Execute binary search
var s=new search();
var arr = [1, 3, 5, 7, 8, 9, 12];
var key = 22;
s.binary_search(arr, key);
```

Output:

```
Element not found!
```

Q.14>Implementation of Dynamic Programming,lcs,Regular Expression Matching.

```
let re;
// re = /hello/;
re = /hello/ig; // 'i' is use make RegExp case insensitive
// re = /hello/g; // 'g' is global for whole pARagraph

const str = 'Hello There Hello';
const result =str.replace(re,'Hi')
console.log(result);
```

Output:

```
Hi There Hi
```

Q.15>Practical based on backtracking -N Queen's Problems.

```
function NQueens(order) {
  if (order < 4) {
    console.log('N Queens problem apply for order bigger than 3 ! ');
    return;
  }

  var nQueens = [];
  rowLoop:
  for (var row=0; row<order; row++) {
    // If appear row Less than 0, There is no solution
    if(row < 0){
      console.log('This N Queens problem has no solution ! ');
      break;
    }
    // The first 1 A new one was detected this time 1 line
    if (nQueens[row] === undefined) {
      nQueens[row] = [];
    }
    // A block of programs that run while backtracking
    for (var col=0; col<order; col++) {
      //0 For the position that has been tested and can place the queen
      if (nQueens[row][col] === 0) {
        continue;
      }
      // In the process of backtracking, the position where the queen can be put is
      encountered, which means that the verification of this position in the future has not
      passed and needs to be processed again
      else if (nQueens[row][col] == 1) {
        // Back in time , on 1 The end of the line , I need to keep going back
        if (col === order-1) {
          resetRow(nQueens, order, row);
          row = row - 2;
          continue rowLoop;
        }
        // The backtracking line is not at the end of the line , mark 0, Continue
        to
        nQueens[row][col] = 0;
        continue;
      }
      // Place the 1 A queen
      nQueens[row][col] = 1;
      // find 1 The queen can be placed in the position, jump down 1 Line ( 1 Only
      on the line 1 A queen).
      if (isQueenValid(nQueens, row, col)) {
        continue rowLoop;
      }
      // every 1 There should be 1 The queen, to the end of the column has not
      found a suitable position, indicating that the queen before the placement of a
      problem, need to backtrack!
      else if (col == order-1) {
        //0 with 1 All indicate that this location has been detected, so clear the
        line as undefined
        resetRow(nQueens, order, row);
        // Reduction of 2 Because the end of the loop has a self addition, which is
        essentially going back up 1 line
        row = row - 2;
        // Step back into the outer loop and continue
        continue rowLoop;
      } else {
        // If the line is not reached, continue to detect the undetected
        nQueens[row][col] = 0;
        continue;
      };
    }
  }
  return nQueens;
}
// Back before , Clear line
function resetRow(nQueens, order, row) {
  for (var col=0; col<order; col++) {
    nQueens[row][col] = undefined;
  }
}
// Check if the position can place the queen
function isQueenValid(nQueens, row, col) {
  // Line detection
  for (var i=0; i<col; i++) {
    if (nQueens[row][i] == 1) {
```

```
        return false;
    }
}
for (var j=1; j<row+1; j++) {
    //      Column tests      Upper left 45 The degree of      The
upper right 45 The degree of
    if (nQueens[row-j][col]==1 || (nQueens[row-j][col-j]==1) || (nQueens[row-
j][col+j]==1)) {
        return false;
    }
}
return true;
}

function printQ(queens) {
    for (var row=0; row<queens.length; row++) {
        var rowText = '';
        for (var col=0; col<queens.length; col++) {
            if (queens[row][col]==undefined) {
                queens[row][col] = 0;
            }
            rowText = rowText + queens[row][col] + ' ';
        }
        console.log(rowText);
    }
}

var queens = NQueens(8);
printQ(queens);
```

output:

```
1 0 0 0 0 0 0 0
0 0 0 0 1 0 0 0
0 0 0 0 0 0 0 1
0 0 0 0 0 1 0 0
0 0 1 0 0 0 0 0
0 0 0 0 0 0 1 0
0 1 0 0 0 0 0 0
0 0 0 1 0 0 0 0
```